

환자의 의료 이해도 향상을 위한
진료 대화 자동 요약 및
의학 용어 설명 앱

그로쓰 산학 트랙 **04팀 MedExplain**

이화여자대학교 엘텍공과대학 소프트웨어학부
컴퓨터공학과

2276285 정아윤

2376190 이다은

2376245 임하령

지도교수 윤 명 국

1. Team Info (팀 정보)

1.1. 과제명

MedExplain: 환자의 의료 이해도 향상을 위한 진료 대화 자동 요약 및 의학 용어 설명 앱

1.2. 팀 정보

그로쓰 산학 트랙 04팀 MedExplain

1.3. 팀 구성원

- 정아윤 (**Back-end**): 서버 인프라 구축 및 시스템 가용성 확보를 담당한다. Google Streaming STT API 세션을 관리하며 서버와 STT 엔진 간의 안정적인 파이프라인을 유지하는 역할을 수행한다. 또한, 비정형 음성 및 텍스트 데이터를 가공하고 구조화된 JSON 데이터로 변환하는 서버 측 파싱 엔진 개발을 주도한다.
- 이다은 (**Front-end**): Flutter 기반의 모바일 인터페이스 구축 및 프론트엔드 안정화를 담당한다. WebSocket을 통한 Audio Chunk의 안정적 전송 로직을 구현하고, 서버 응답 처리를 최적화하여 실시간 데이터 스트리밍 인터페이스를 구축한다. 안드로이드 및 iOS 플랫폼별 오디오 권한 제어와 백그라운드 스트리밍 안정화 작업을 수행한다.
- 임하령 (**UX/Presentation**): 사용자 인지 과정을 고려한 실시간 정보 시각화 인터랙션 설계 및 화면 흐름 기획을 담당한다. 실제 환자 진료 시나리오를 바탕으로 시스템의 완성도를 평가할 수 있는 검증 모델을 설계하며, 문서 및 발표 자료 제작을 주도한다.

2. Project-Summary (과제 요약)

2.1. 문제 정의

MedExplain은 현대 의료 시스템에서 정보 소외가 가장 빈번하고 심각하게 발생하는 장기 입원 환자들이 처한 불안정한 소통 환경을 배경으로 한다. 어려운 수술이나 중증 질환 치료를 앞둔 환자에게는 복잡한 치료 과정과 앞으로의 관리 절차를 온전히 파악하기 어려운 정보의 공백기가 발생한다. 이러한 배경 속에서 정의된 목표 사용자(Target Customer)는 복잡한 의료 정보를 수용하기 어려운 장기 입원 환자와, 환자를 대신하여 의사의 설명과 환자의 상태를 완벽히 이해하고 환자를 케어해야 하는 보호자이다. 이들이 의료 현장에서 겪는 핵심적인 Pain Point는 다음과 같이 크게 세 가지로 요약된다.

첫째, 의료 전문 용어가 만드는 심리적 장벽과 인지적 고립이다. 의료진이 사용하는 의학 용어는 일상 언어와 동떨어져 있어, 환자는 자신의 상태에 대한 설명을 들어도 그 실질적인 의미를 파악하기 어렵다. "CRP 수치가 급격히 상승했다"거나 "크레아티닌 수치가 불안정하여 투여량을 조절한다"와 같은 의학적 표현은 일반 환자에게 막연한 공포감을 줄 뿐, 자신의 신체 상태가 구체적으로 어떻게 변화하고 있는지에 대한 실질적인 이해를 돕지 못한다. 이러한 언어적 장벽은 환자가 치료 과정에서 능동적인 주체가 아닌 수동적인 객체로 머물게 만든다.

둘째, 회진 정보의 극심한 휘발성이다. 일반 병동의 회진은 짧고 빠르게 진행되는 특성상 환자가 긴장감 속에서 의사가 전달하는 수치 변화나 처방 변경의 이유를 실시간으로 기록할 여유가 전혀 없다. 이는 진료실 문을 나서는 순간 환자가 "무엇을 조심해야 하는지" 혹은 "약이 왜 바뀌었는지"를 잊게 만드는 결정적 원인이 되며, 결국 사후 관리의 치명적인 공백을 초래한다.

셋째, 진료 경과를 지속적으로 확인할 수 있는 환자용 기록 수단의 부재이다. 장기 입원 환자는 매일 변하는 검사 수치와 치료 경과를 정확히 인지해야 하지만, 일반적인 의료 기록 서비스들은 의료진의 기록 편의성에 집중되어 있어 정작 환자는 자신의 상태가 호전되고 있는지, 혹은 어떤 지표를 주의 깊게 살펴야 하는지에 대한 중요한 정보를 놓치게 된다.

MedExplain에서는 이러한 의료 현장의 문제점을 기술적으로 보완하여 환자를 위한 의료 케어 환경을 구축하고자 한다.

2.2. 기존 연구와의 비교

현재 의료 시장에서 주목받고 있는 글로벌 의료 AI 서비스들과의 상세한 비교 분석을 통해, MedExplain만이 가지는 독창성과 환자 중심의 차별적 우위를 다음과 같이 기술한다.

- **Nuance Dragon Medical One:** 마이크로소프트의 자회사인 Nuance가 제공하는 이 서비스는 클라우드 기반의 음성 인식 솔루션으로, 의사가 진료 내용을 말하면 이를 실시간으로 텍스트화하여 전자의무기록(EMR)에 입력해 주는 기능을 수행한다. 의료진의 차트 작성 시간을 획기적으로 단축하고 행정 업무의 효율성을 극대화한다는 점이 있으나, 철저히 의료진 전용으로 설계되어 환자가 자신의 진료 경과를 직접 확인하거나 어려운 용어를 해설받는 기능이 전혀 포함되어 있지 않다는 한계가 있다. 반면 MedExplain은 환자를 주요 사용자로 설정하여 환자의 이해를 돕는 리포트 생성에 집중한다.
- **Amazon HealthScribe:** 아마존 웹 서비스(AWS)에서 제공하는 이 엔진은 환자와 의사 간의 대화를 분석하여 임상 요약본(Clinical Note)을 자동으로 생성하는 생성형 AI 기반 서비스이다. 대화 내용을 구조화하는 성능이 뛰어나고 병원 시스템 관리와의 연동이 용이하다는 점이 있지만, 주요 목적이 의사의 업무 효율화에 초점이 맞춰져 있어 산출되는 문서가 여전히 전문적이고 환자가 읽기에는 가독성이 떨어진다. MedExplain은 이와 달리 Gemini AI를 통해 전문 용어를 환자의 눈높이에 맞춘 '쉬운 말'로 재구성하여 제공한다는 점이 핵심적인 차별점이다.
- **Suki Ai:** 의료진을 위한 AI 음성 비서로서 진료 기록의 정확성을 높이고 음성 명령을 통해 EMR을 조작하는 데 특화되어 있다. 진료 현장에서의 음성 분석 및 데이터 정확도가 높으나, 중환자나 장기 입원 환자가 겪는 정보 소외 문제를 해결하기 위한 환자 공유용 인터페이스나 지속적인 상태 경과 추적을 위한 타임라인 기능이 결여되어 있다. MedExplain은 환자가 직접 자신의 치료 과정을 시각적인 타임라인으로 관리할 수 있는 기능을 제공하여 환자의 주체적인 의료 정보 관리를 지원한다.

2.3. 제안 내용

MedExplain은 앞서 분석한 장기 입원 환자와 보호자의 **Pain Point**를 근본적으로 해결하기 위해 각각의 **Pain Point**와 매칭된 세 가지 핵심 솔루션을 다음과 같이 제안한다.

첫째, 회진 정보의 휘발성 문제를 해결하기 위해 회진 현장의 음성을 기반으로 진료 내용을 기록하고 자동으로 구조화하여 요약한다. 환자가 의사의 설명을 놓치지 않도록 **STT** 기술을 활용해 실시간으로 대화를 텍스트화하며, 이를 **LLM** 엔진이 즉각적으로 분석하여 "오늘의 상태 변화", "조절된 약물 목록", "내일의 검사 계획" 등 환자가 반드시 알아야 할 핵심 정보 위주로 구조화된 요약을 생성하여 제공한다.

둘째, 전문 용어의 장벽을 해결하기 위해 의료 용어를 탐지하여 일상어로 변환한다. 텍스트 내에서 전문 용어를 식별하고, 이를 "염증 수치"나 "신장 기능"과 같이 환자가 평소 사용하는 쉬운 말로 자동 변환하여 ~~보충 설명과 함께 노출한다.~~ 식별된 용어는 환자가 터치하여 즉시 확인할 수 있는 인터랙티브 카드로 제공되어 의료 이해도를 실질적으로 향상시킨다. 이는 환자가 자신의 질병 상태를 왜곡 없이 이해하고 치료에 대한 두려움을 극복하게 하는 핵심 장치가 된다.

셋째, 환자용 기록을 위한 타임라인 기반 진료 리포트를 제공한다. 생성된 모든 요약 리포트와 용어 설명 데이터를 개인별 진료 기록으로 자동 저장하고, 이를 시간 순서에 따른 타임라인 뷰로 시각화한다. 이를 통해 환자는 어제와 오늘의 진료 내용이 어떻게 달라졌는지, 상태가 호전되고 있는지 타임라인을 통해 스스로 확인하며 능동적으로 건강을 관리할 수 있으며, 보호자도 그동안의 회진 기록을 한눈에 확인하며 환자의 상태를 명확히 파악하고 환자 케어에 적극 참여할 수 있게 된다.

2.4. 기대 효과 및 의의

본 과제는 환자가 자신의 의료 정보를 주도적으로 파악하고 관리하게 함으로써 의료진과 환자 사이의 정보 격차를 비약적으로 최소화하는 데 그 필요성이 있다. 병원 진료 과정에서 전달되는 정보는 방대하지만 환자가 이를 정확히 기억하고 기록하기 어려운 현실에서, 환자가 자신의 진료 내용을 직접 이해하고 경과를 관리할 수 있는 도구는 필수적인 요소이다.

복잡한 수치 변화와 진료 설명을 데이터로 축적하여 제공하는 것은 환자의 알 권리를 보장한다는 의의를 지니는 동시에 단순한 기록의 의미를 넘어 환자 본인의 상태에 대한 이해도를 근본적으로 향상시키는 효과를 거둔다. 또한 보호자 역시 환자 곁을 24시간 지키지 못하더라도 시스템에 기록된 상세한 타임라인 리포트를 통해 장기적인 환자 케어에 가담할 수 있다. 결과적으로 본 과제의 결과물은 의료진의 설명 의무를 기술적으로 보완함으로써 미래 지향적인 환자 중심 디지털 의료 환경을 선도하는 모델이 될 것이다.

2.5. 주요 기능 리스트

MedExplain의 솔루션을 실현하기 위한 구체적인 기능 리스트와 상세 설명은 다음과 같다. 첫째, **WebSocket** 기반의 실시간 음성 전사(STT) 기능이다. 환자가 회진이나 진료 시 녹음 버튼을 활성화하면, 모바일 기기를 통해 수집된 음성 데이터가 실시간 오디오 청크(Audio Chunk) 단위로 서버에 전송된다. 서버는 **Google Streaming STT** 기술을 활용하여 이를 즉각적인 텍스트로 변환하며, 특히 한국어 조사와 영어 의학 약어가 뒤섞인 복잡한 의료 현장의 특수성을 고려하여 핵심 단어를 정확히 추출하는 하이브리드 엔진을 가동한다. 또한 **TranscriptStore** 모듈을 통해 "음, 아, 어" 등 한글 필러를 실시간 제거하여 가독성을 극대화한다. 이를 통해 화면에 텍스트가 실시간으로 렌더링되어 환자가 진료 도중 대화의 흐름을 시각적으로 동기화할 수 있는 환경을 구축한다.

둘째, 대규모 언어 모델(LLM)을 활용한 진료 대화 구조화 및 요약 기능이다. 전사된 비정형 대화 데이터는 **Gemini AI** 엔진으로 전달되어 정밀한 맥락 분석 과정을 거친다. 단순히 문장을 요약하는 수준을 넘어, 사전에 설계된 시스템 프롬프트를 통해 '주요 증상', '검사 결과 수치', '변경 약물', '주의사항' 등의 핵심 엔티티(Entity)를 추출한다. 이 과정에서 사용자가 AI의 요약 결과를 직접 편집하거나 개인 메모를 추가할 수 있는 편집 인터페이스를 지원한다. 특히 장기 입원 환자에게 중요한 'CRP 수치'나 '복용 시점'과 같은 수치형 데이터를 정밀하게 포착하여 환자용 요약 리포트에 구조화된 형태로 배치한다.

셋째, 자연어 처리(NLP) 기반 의료 전문 용어 자동 탐지 및 일상어 변환 기능이다. 시스템은 전사된 텍스트 내에서 'CRP 수치', '절제술', '전이'와 같은 고도의 전문 의학 용어를 자동으로 식별하기 위해 자연어 처리(NLP) 알고리즘을 가동한다. 식별된 용어는 즉시 일반 환자가 이해하기 쉬운 표현(예: "염증 수치", "암세포가 퍼짐")으로

매핑되며, AI가 전체 문맥에 맞는 보충 설명을 생성하여 제공한다. 이 과정에서 의료 도메인에 특화된 프롬프트를 활용하여 용어의 오인식을 방지하고, 문맥상 해당 용어가 환자의 현재 상태와 어떻게 연결되는지를 기술적으로 판단한다.

넷째, 환자 중심의 타임라인 기록 관리 및 리포트 생성 기능이다. 생성된 모든 진료 요약과 용어 설명 데이터는 정형화된 **JSON** 포맷으로 변환되어 데이터베이스에 자동 저장된다. 저장된 데이터는 환자별 고유 ID와 매핑되어 시간 순서(**Timeline**)에 따라 시각화되며, 과거의 진료 경과와 현재 상태를 비교 분석할 수 있는 기반을 제공한다. 이러한 히스토리 관리는 만성 질환 관리나 반복되는 병원 방문 상황에서 환자가 자신의 의료 정보를 지속적으로 확인하고 관리하며 의료 주권을 확보하는 데 결정적인 역할을 수행한다.

다섯째, 진료과 자동 추천 및 지능형 **Q&A** 기능이다. 전사된 진료 내용을 분석하여 내과, 정형외과 등 사전 정의된 **14개** 주요 진료과 중 최적의 카테고리를 적용하여 리포트의 분류 편의성을 높인다. 또한 **Q&A** 기능은 환자가 진료 원문을 바탕으로 궁금한 점을 질문하면 AI가 답변을 생성하며, 이 중 별도로 의사의 재확인이나 조치가 필요한 부분을 문맥에서 식별하여 빨간 경고 아이콘과 함께 제시함으로써 진료 과정에서 환자의 추가 질문을 지원한다.

3. Project-Design (과제 설계)

3.1. 요구사항 정의

본 프로젝트의 목적은 중환자 및 장기 입원 환자가 직면하는 의료 정보의 불균형을 해소하는 것이며, 이를 위해 시스템이 반드시 갖추어야 할 기능적·기술적 요구사항을 다음과 같이 상세히 정의한다.

첫째, 실시간성 및 낮은 지연 시간(**Low Latency**) 보장이다. 진료실 내에서 의사와 환자의 대화가 이루어지는 즉시 정보가 시각화되어야 하므로, 음성 입력부터 텍스트 출력까지의 지연 시간을 **5초** 이내로 유지해야 한다. 이를 위해 클라이언트와 서버 간의 양방향 통신을 지원하는 **WebSocket** 프로토콜 도입이 필수적이며, 오디오 데이터를 잘게 쪼개진 청크(**Chunk**) 단위로 스트리밍 전송하여 처리 효율을 극대화해야 한다.

둘째, 의료 도메인 특화 데이터 처리 및 요약 능력이다. 단순한 일상 대화 녹음기가 아니기에, 한국어 조사와 영어 의학 약어(예: **CRP**, **Creatinine** 등)가 혼용된 복잡한 문장 구조에서도 핵심 단어를 정확히 추출해야 한다. LLM(Gemini AI)을 활용하여 비정형 대화에서 진단명, 처방 약물, 향후 검사 일정 등 환자가 반드시 알아야 할 '핵심 엔티티'를 선별하고, 이를 구조화된 리포트 형태로 재구성하는 기능이 요구된다.

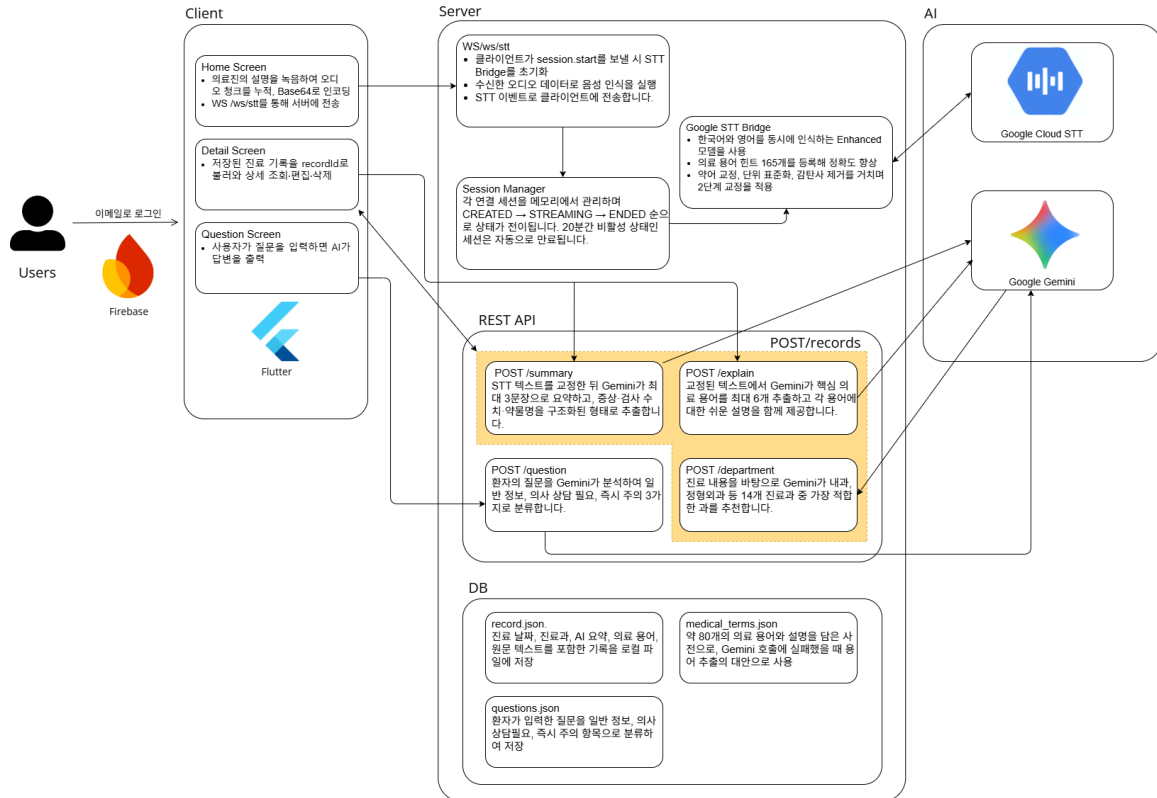
셋째, 환자 중심의 인터랙티브 인터페이스 구현이다. 전문 용어를 단순히 텍스트로 나열하는 것이 아니라, 사용자가 해당 용어를 터치했을 때 즉각적으로 쉬운 해설을 제공하는 인터랙티브 카드를 구현해야 한다. 또한, 장기 입원 환자의 특성을 고려하여 날짜별, 진료과별로 자신의 상태 변화를 한눈에 추적할 수 있는 타임라인 뷰(Timeline View)를 통해 의료 주권을 확보할 수 있는 대시보드 기능을 갖추어야 한다.

넷째, 데이터의 안정성과 보안성 확보이다. 민감한 의료 데이터를 다루는 만큼, 스트리밍 세션 관리 프로토콜을 상세히 설계하여 네트워크 불안정 시에도 데이터 유실 없이 대화 맥락을 유지해야 한다. 모든 통신은 보안 프로토콜을 거치며, 최종적으로 생성된 리포트는 정형화된 데이터 포맷(JSON)으로 변환되어 개인별 히스토리에 안전하게 영속화되어야 한다.

3.2. 전체 시스템 구성

MedExplain의 시스템 아키텍처는 사용자(환자/보호자)와 의료진 간의 소통을 기술적으로 중재하기 위해 클라이언트, 통신 계층, 서버 엔진, 그리고 데이터 계층이 유기적으로 연결된 다층 구조로 설계되었다.

[전체 시스템 구조]



전체 SW 구조 설명

백엔드는 FastAPI 기반의 Python 서버로, 크게 세 레이어로 구성된다.

1. WebSocket Handler 레이어

Flutter 앱에서 전송되는 오디오 스트림을 WebSocket으로 수신하고, Google Cloud STT에 전달하여 실시간 음성 인식을 처리한다. 인식된 텍스트는 후처리를 거쳐 앱으로 반환된다.

2. REST API Routes 레이어

녹음 완료 후 앱이 호출하는 HTTP 엔드포인트를 담당한다. /summary(요약), /explain(용어 추출), /records(진료 기록 CRUD), /questions(질문 분류) 네 가지 라우터로 구성된다.

3. Services 레이어

실제 비즈니스 로직이 구현된 모듈들의 집합이다. 각 모듈은 독립적으로 동작하며 **Routes** 레이어에서 호출된다. **Gemini API**와의 통신도 이 레이어에서 처리된다.

[데이터 흐름 상세 설명]

1단계: 진료 준비 및 Q&A 단계 (Flutter Client)

시스템의 접점인 **Flutter** 기반 모바일 애플리케이션은 환자와 보호자가 진료의 주체가 될 수 있는 환경을 제공한다. 사용자는 진료실에 들어가기 전, 이전 진료 내용이나 현재 환자의 상태에 대해 궁금한 점을 앱 내 **AI**에게 먼저 자유롭게 질문하여 기초적인 답변을 얻을 수 있다. 이 과정에서 **AI**의 답변만으로 부족하거나 의사의 직접적인 확인이 꼭 필요한 사항들은 별도의 질문 목록으로 선택하여 보관한다. 시스템은 이 질문들을 진료 중 환자가 바로 꺼내 볼 수 있도록 화면 상단에 상시 배치하며, 사용자가 설정한 서버 접속 정보를 바탕으로 실시간 소통을 위한 통신 세션을 초기화한다. 이 단계에서 **UI**는 복잡한 의료 환경에서도 직관적으로 사용할 수 있도록 설계되었으며, 사용자의 조작에 따라 후속 단계인 실시간 음성 스트리밍 세션을 시작할 준비를 마친다.

2단계: 실시간 음성 인식 및 전사 단계 (WebSocket & STT)

진료가 시작되면 사용자의 스마트폰 마이크를 통해 수집된 **PCM** 형식의 음성 데이터는 **WebSocket** 프로토콜을 통해 실시간 오디오 청크 단위로 백엔드 서버에 전송된다. 서버의 **STT** 처리 모듈(**STT Processing Module**)은 수신된 음성 스트림을 외부 엔진인 **Google Streaming STT API**로 즉각 전달한다. 변환된 텍스트 데이터는 다시 **WebSocket** 이벤트를 통해 클라이언트로 송출되어, 사용자는 의사의 설명을 귀로 듣는 동시에 화면에 나타나는 실시간 텍스트를 눈으로 읽으며 정보의 유실을 방지한다.

3단계: 지능형 진료 요약 및 맥락 분석 단계 (Summary Module)

녹음이 종료되면 서버의 요약 모듈(**Summary Module**)은 수집된 전체 진료 텍스트를 취합하여 **Gemini API**로 전달한다. 이때 시스템은 사전에 최적화된 프롬프트 구조를 활용하여 비정형 대화에서 진단 내용, 처방 약물, 향후 검사 일정 등 환자에게 치명적일 수 있는 핵심 정보를 선별한다. 분석이 완료되면 서버는 요약된 리스트를 클라이언트로 전송하며, 앱 화면에서는 각 항목이 편집 가능한 텍스트 상자로

변환되어 사용자가 내용을 직접 수정하거나 추가 입력을 할 수 있는 상태로 대기한다. 단순히 문장을 줄이는 것을 넘어 '오늘 진료의 핵심'이라는 명확한 구조로 정보를 재구성함으로써, 환자가 방대한 설명 속에서도 가장 중요한 권고 사항을 직관적으로 파악할 수 있도록 돕는다.

4단계: 의료 용어 탐지 및 환자 친화적 해설 생성 단계 (Medical Term Module)

환자가 가장 큰 장벽을 느끼는 전문 의학 용어를 해결하기 위해 자연어 처리(NLP) 기반의 용어 해설 모듈이 작동한다. 시스템은 텍스트 내에서 'CRP 수치', '크레아티닌' 등 고도의 전문 용어를 자동으로 식별하고, 이를 Gemini API의 지능형 판단을 통해 "염증 수치"나 "신장 기능 지표"와 같은 환자 친화적인 일상어로 자동 변환한다. 탐지된 용어는 앱 화면에서 클릭 가능한 인터랙티브 객체로 렌더링되며, 클릭 시 상세 설명을 제공하여 환자의 의료 이해도를 실질적으로 향상시킨다.

5단계: 데이터 영속화 및 타임라인 기록 관리 단계 (Database & API)

최종 가공된 요약 리포트와 용어 해설 데이터는 정형화된 JSON 포맷으로 변환되어 데이터베이스(DATABASE)에 안전하게 저장된다. 저장 직전 시스템은 전체 대화 맥락을 분석하여 해당 진료가 내과인지 정형외과인지 등의 적절한 진료 카테고리를 먼저 제안해준다. 환자가 이를 최종 승인하면 진료 날짜와 메모가 포함된 리포트가 타임라인에 저장되며, 이후 언제든지 과거의 건강 변화를 추적할 수 있는 상태가 된다. 저장된 기록은 사용자의 고유 식별자와 매핑되어 시간 순서에 따른 타임라인 형태로 시각화된다. 환자와 보호자는 진료 기록 관리 모듈(Record Management Module)을 통해 과거의 진료 경과를 언제든지 조회하고 추적할 수 있으며, 이를 통해 만성 질환 관리나 반복되는 병동 생활에서 자신의 의료 정보를 지속적으로 확인하며 의료 주권을 확보하게 된다.

[모듈별 역할]

모듈	역할
stt_google_streaming.py	Google STT 스트리밍 브릿지 및 오디오 후처리
stt_corrector.py	STT 오인식 교정 (규칙 기반 + Gemini LLM)
summarizer.py	Gemini 기반 의료 요약 및 구조화 정보 추출
term_extractor.py	Gemini 기반 의료 용어 추출 및 설명
question_analyzer.py	Gemini 기반 질문 자동 분류 및 AI 답변 생성

- 1. Client Layer (Flutter Mobile App):** 사용자가 직접 마주하는 인터페이스로, 스마트폰 마이크를 통해 오디오 데이터를 수집한다. 수집된 PCM 형식의 오디오 데이터는 실시간 스트리밍 최적화를 위해 청크 단위로 분할되며, **WebSocket** 클라이언트를 통해 서버와 상시 연결을 유지한다. 서버로부터 반환된 실시간 전사 결과와 최종 요약 리포트, 인터랙티브 용어 카드를 화면에 렌더링하여 사용자에게 직관적인 경험을 제공한다.
- 2. Communication Layer (WebSocket & FastAPI):** 고성능 비동기 프레임워크인 **FastAPI**를 기반으로 서버를 구축하여 대량의 스트리밍 데이터를 저지연으로 처리한다. **WebSocket** 프로토콜은 오디오 데이터의 업로드와 전사 결과의 다운로드를 동시에 수행하는 양방향 통신을 담당하며, 세션 관리를 통해 안정적인 데이터 파이프라인을 유지한다.
- 3. Intelligence Engine Layer (External Module Integration):**
 - **Google Streaming STT API:** 서버로 전송된 오디오 청크를 실시간으로 텍스트화한다. 중간 결과(Interim Results) 반환 기능을 활용해 의사의 말이 끝나기 전에도 텍스트를 미리 노출하여 실시간성을 확보한다.
 - **Gemini 1.5 Pro (LLM Engine):** 시스템의 핵심 두뇌로, 전사된 전체 텍스트를 분석한다. 프롬프트 엔지니어링 기술을 적용하여 의료 용어를 탐지하고, 일상어(예: CRP → 염증 수치)로의 변환 및 문맥에 맞는 보충 설명을 생성한다. 또한 비정형 데이터를 정형 **JSON** 포맷으로 구조화하여 리포트의 완성도를 높인다.
- 4. Data Layer (Storage & Persistence):** 처리된 모든 진료 데이터와 요약 리포트는 정형화된 형태로 저장된다. 이는 환자용 타임라인 아카이브의 기반이 되며, 사용자가 과거 특정 시점의 진료 경과와 의사의 권고 사항을 다시 조회할 때 빠르고 정확한 데이터 매핑을 지원한다.

[현재까지의 개발 현황 및 검증 내용] ~~현재 프로젝트는 핵심 파이프라인의 엔드 투 엔드(End-to-End) 구축을 완료하고 성능 고도화 단계에 있다.~~

주요 엔진 기능에 있어서는 **WebSocket** 기반의 실시간 오디오 스트리밍 파이프라인을 구축하여 **Flutter** 앱과 백엔드 서버 간의 양방향 통신 및 오디오 청크 전송 로직이 정상 작동함을 확인하였다.

데이터 파싱 및 구조화에 있어서는 비정형 대화 데이터를 **LLM**을 통해 분석하고, 이를 다시 정형화된 **JSON** 데이터로 변환하는 파싱 로직을 구현 완료하였다. 이를 통해 진단명, 약물, 일정 등의 핵심 정보를 누락 없이 추출할 수 있는 상태이다.

현재 실시간 전사와 맥락 분석 결과 반환까지의 통합 엔진 구축이 완료되었으며, 프론트엔드에서 발생하는 이벤트(**translate** 이벤트)의 분기 처리 및 **UI** 최적화 작업을 진행하고 있다. 환자는 진료 종료와 거의 동시에 자신만을 위한 '디지털 진료 리포트'를 스마트폰으로 받아보게 될 것이며, 이는 정보 소외 문제를 획기적으로 해결하는 결과물로 이어질 것이다.

WebSocket 기반 양방향 통신 및 **Gemini** 기반 정형 **JSON** 파싱 엔진 구축을 완료하였으며, 현재 **UI** 최적화 및 **Q&A** 질문 탐지 로직의 통합 테스트를 진행 중이다.

3.3 주요 엔진 및 기능 설계

3.3.1 STT 파이프라인 및 교정 파이프라인

[Backend]

진료 음성을 텍스트로 변환하는 과정에서 두 가지 문제가 발생한다. 첫째, **Google STT**가 의료 용어를 구어체로 인식하는 문제(예: "에이치비에이원씨" → **HbA1c**)이고, 둘째, 알파벳과 숫자를 혼동하는 문제(예: "**L4 LO**" → **L4 L5**)이다. 이를 해결하기 위해 **stt_google_streaming.py**의 후처리 함수와 **stt_corrector.py**의 교정 파이프라인을 단계적으로 적용한다.

stt_google_streaming.py: STT 설정 및 후처리

Google STT 인식률을 높이기 위해 **SpeechContext**에 의료 용어 힌트 목록을 추가하고 **boost=25.0**으로 가중치를 설정한다. 인식 완료(**is_final=True**) 시점에 **postprocess_stt_text()**를 적용한다.

```
def _medical_phrase_hints() -> list[str]:
```

```

return [
    "CRP", "CBC", "MRI", "HbA1c", "eGFR",
    "위선암", "내시경", "항암화학요법", "당화혈색소", ...
]

def _build_recognition_config(sample_rate, channels):
    speech_context = speech.SpeechContext(
        phrases=_medical_phrase_hints(),
        boost=25.0,          # 해당 단어 인식 시 가중치 증가
    )
    return speech.RecognitionConfig(
        language_code="ko-KR",
        alternative_language_codes=["en-US"], # 영어 의료용어 병행 인식
        model="latest_long",
        use_enhanced=True,
        speech_contexts=[speech_context],
    )

```

postprocess_stt_text()는 4가지 처리를 순서대로 수행한다.

```

def postprocess_stt_text(text: str) -> str:
    # 1. 의료 약어 구어체 교정: "씨알피" → "CRP"
    for ko, en in _ABBR_CORRECTIONS.items():
        text = re.sub(ko, en, text, flags=re.IGNORECASE)

    # 2. 단위 구어체 표준화: "밀리그램" → "mg"
    for ko, unit in _UNIT_CORRECTIONS.items():
        text = re.sub(ko, unit, text)

    # 3. 소수점 구어체 변환: "영점오" → "0.5"
    text = _DECIMAL_PATTERN.sub(_fix_decimal, text)

    # 4. 간투사 제거: "음", "어", "그" 등

```

```
text = _FILLER_PATTERN.sub("", text)
```

```
return text
```

stt_corrector.py: 2단계 교정 파이프라인

후처리로 잡히지 않는 오류를 처리하는 별도 모듈이다. 규칙 기반 교정을 먼저 수행하고, LLM 교정을 후속으로 적용한다.

1단계: 척추 레벨 등 예측 가능한 패턴은 정규식으로 즉시 교정한다.

```
_SPINAL_RULES = [  
    (re.compile(r'\bL([1-4])\s+L[000]\b'), r'L\1 L5'),    # L4 LO -> L4 L5  
    (re.compile(r'\bL[000]\b'), 'L5'),                  # 단독 LO -> L5  
    (re.compile(r'\bl([1-5])1([1-5])\b', re.IGNORECASE), r'L\1-L\2'), # 1415 -> L4-L5  
]
```

```
def rule_based_correct(text: str) -> str:
```

```
    for pattern, repl in _SPINAL_RULES:
```

```
        text = pattern.sub(repl, text)
```

```
    return text
```

2단계: 규칙으로 처리되지 않는 문맥적 오류는 Gemini에 위임한다. 프롬프트에 "내용 추가/삭제 금지"를 명시하여 의료 정보 왜곡을 방지한다.

```
def llm_correct(text: str) -> str:
```

```
    client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

```
    response = client.models.generate_content(
```

```
        model="gemini-2.5-flash",
```

```
        contents=_build_correction_prompt(text),
```

```
        config=types.GenerateContentConfig(temperature=0.1),
```

```
    )
```

```
corrected = (response.text or "").strip()

return corrected if corrected else text    # 빈 응답 시 원문 반환
```

```
def correct_stt_text(text: str, use_llm: bool = True) -> str:
```

```
    text = rule_based_correct(text)    # 1차: 정규식
```

```
    if use_llm:
```

```
        text = llm_correct(text)    # 2차: Gemini
```

```
    return text
```

[Frontend]

실시간 음성 인식(STT) 기능의 구현을 위하여 record 패키지와 web_socket_channel 패키지를 도입하였고, WsTransport 모듈, RecordingController 모듈, TranscriptStore 모듈을 구현하였다.

WsTransport 모듈의 기능은 WebSocket 연결 관리(A1)와 지수 백오프 자동 재연결(A2)이고, A1은 WebSocketChannel.connect(uri)를 호출한 후 채널 스트림에 리스너를 등록하는 방법으로 구현되었으며, A2는 연결 종료 시 onDone / onError 콜백에서 _scheduleReconnect()를 호출하는 방법으로 구현되었다. WsTransport 모듈은 A1, A2 기능을 수행하는데, A1의 처리 알고리즘은 수신 데이터가 String이면 그대로, binary이면 utf8.decode() 후 WsEvent.tryParse()로 JSON 파싱하고, 파싱에 성공하면 _eventCtrl에 추가하며 실패(null)는 조용히 무시하는 것이다. A2의 처리 알고리즘은 $2 \ll _retryCount$ (비트 시프트)로 대기 시간을 지수적으로 증가시키되 상한은 30초로 고정하고(2→4→8→16→30초), 정상 연결 시 _retryCount를 0으로 초기화하며, close()가 명시적으로 호출되면 내부 _closed = true로 설정하여 재연결 루프를 완전히 종료하는 것이다.

TranscriptStore 모듈의 기능은 STT 이벤트 누적(B1)과 한글 필터 제거(B2)이고, B1은 ChangeNotifier를 상속하여 final 세그먼트 목록(_finalSegments)과 중간 인식 결과(_interim)를 분리 관리한 후 notifyListeners()로 UI에 알리는 방법으로 구현되었으며, B2는 정규식 `RegExp(r"\b(음+|아+|어+|그+|워+|저+|이제)\.{0,3}\b", unicode: true)`를 _removeFiller()에서 replaceAll()로 적용하는 방법으로 구현되었다.

TranscriptStore 모듈은 B1, B2 기능을 수행하는데, B1의 처리 알고리즘은 `apply(SttEvent e)` 호출 시 먼저 B2(필러 제거)를 적용하고, 결과가 빈 문자열이면 조기 반환하며, `isFinal == true`이면 `_finalSegments`에 추가하고 `_interim`을 초기화하고, `isFinal == false`이면 `_interim`을 교체한 후 `notifyListeners()`를 호출하는 것이다. B2의 처리 알고리즘은 단어 경계(`\b`)를 기준으로 매칭하므로 단독 필러(예: "음 드세요"의 "음")는 제거되지만, 단어 내부의 동일 음절(예: "음식"의 "음", "아이스크림"의 "아")은 뒤에 단어 문자가 이어지므로 `\b`가 불일치하여 제거되지 않는 것이다.

RecordingController 모듈의 기능은 녹음 제어(C1), WebSocket 초기화 및 이벤트 라우팅(C2), 오디오 인코딩 및 전송(C3)이다. C1은 `AudioRecorder.startStream(RecordConfig(encoder: pcm16bits, sampleRate: 16000, numChannels: 1))`으로 스트림을 열고 수신 청크를 `_audioChunks` 리스트에 누적하는 방법으로 구현되었으며, C2는 `stateStream`과 `eventStream`에 각각 리스너를 등록하여 상태 변화를 `connState`에 반영하고 수신 이벤트를 `SttEvent` 또는 `WarningEvent`로 파싱하여 `transcriptStore`에 전달하는 방법으로 구현되었으며, C3은 녹음 중지 시 `_audioChunks.expand((x) => x).toList()`로 청크를 합산한 후 `base64Encode(bytes)`로 인코딩하여 WebSocket으로 전송하는 방법으로 구현되었다. RecordingController 모듈은 C1, C2, C3 기능을 수행하는데, C3의 처리 알고리즘은 전송 메시지에 `seq: DateTime.now().millisecondsSinceEpoch`를 포함하여 서버 측 순서 보장을 돕고, 전송 완료 후 `analyzing = bytes.isNotEmpty`로 설정하여 로딩 스피너를 표시하며, 이후 `transcriptStore`에 텍스트가 수신되면 `analyzing = false`로 해제하는 것이다. 생성자에서 `transcriptStore.addListener(notifyListeners)`를 등록하여 TranscriptStore의 변화가 자동으로 상위 UI에 전파되도록 하였다.

3.3.2 API 통신 계층

REST API 통신 기능의 구현을 위하여 `http` 패키지와 `shared_preferences` 패키지를 도입하였고, ApiService 모듈과 AppConfig 모듈을 구현하였다.

ApiService 모듈의 기능은 REST API 요청 처리(D1)와 에러 처리(D2)이고, D1은 `http.Client`를 인스턴스 수준에서 공유하는 Singleton 패턴으로 구현되었으며(`static final ApiService instance = ApiService._()`), D2는 HTTP 응답 상태코드가 2xx 범위

밖이면 `ApiException(statusCode, message)`을 throw하는 `_checkStatus()` 메서드로 구현되었다. `ApiService` 모듈은 D1, D2 기능을 수행하는데, D1의 처리 알고리즘은 모든 POST 요청에 `Content-Type: application/json; charset=utf-8` 헤더를 명시하고, 응답 본문은 `response.body` 대신 `utf8.decode(response.bodyBytes)`로 디코딩하여 한글 깨짐을 방지하며, 각 응답은 대응하는 데이터 클래스의 `fromJson()` factory 생성자로 역직렬화하는 것이다. 서버 URL은 매 호출마다 `await getApiBaseUrl()`로 동적으로 취득하므로 런타임 중 서버 IP 변경이 즉시 반영된다.

`AppConfig` 모듈의 기능은 서버 IP 관리(E1)이고, E1은 메모리 캐시(`_cachedIp` 변수)와 `SharedPreferences`를 2단계로 사용하는 방법으로 구현되었다. `AppConfig` 모듈은 E1 기능을 수행하는데, E1의 처리 알고리즘은 `getServerIp()` 호출 시 먼저 `_cachedIp`가 null이 아니면 즉시 반환하고, null이면 `SharedPreferences`에서 읽은 후 `_cachedIp`에 캐싱하며, 그것도 없으면 기본값(10.240.66.72)을 반환하는 것이다. `saveServerIp()`는 메모리와 디스크를 동시에 갱신하여 이후 모든 API/WebSocket 요청에 즉시 새 IP가 적용되도록 한다.

3.3.3 Gemini 기반 의료 요약 모듈

[Backend]

교정된 텍스트를 환자가 이해할 수 있는 언어로 요약하는 모듈이다. 단순 원문 축약이 아니라 환자 언어로 재구성하되, 수치는 정확하게 유지하는 것이 핵심 설계 원칙이다.

프롬프트 설계

요약 품질을 결정하는 핵심은 프롬프트 설계이다. 아래 두 가지 규칙이 가장 중요하다.

- 수치는 반드시 괄호로 병기 (예: "혈당이 당뇨 직전 수준이다 (공복혈당 128, HbA1c 6.8%)")
- 원문에 없는 내용 추가 금지 (의료 정보 왜곡 방지)

```
def _build_prompt(text: str) -> str:
```

```
    return f"""
```

너는 환자가 진료 내용을 이해할 수 있도록 돕는 요약 도우미다.

요약 방식:

- 원문을 그대로 줄이는 것이 아니라, 핵심 정보를 환자 언어로 재구성할 것
- 의학 용어는 쉬운 말로 풀되, 실제 수치는 반드시 괄호로 병기할 것
(예: "혈당이 당뇨 직전 수준이다 (공복혈당 **128**, HbA1c **6.8%**)")
- 환자에게 중요한 순서로 작성 (진단/결과 -> 치료 방향 -> 주의사항)

지킬 규칙:

1. 원문에 없는 내용은 추가하지 말 것
2. 최대 3개 **bullet**
3. 출력은 **JSON**만 할 것

출력 형식:

```
{{  
  "summary": ["문장1", "문장2", "문장3"],  
  "symptoms": ["증상1"],  
  "measurements": ["HbA1c 6.8%"],  
  "medications": ["메트포르민 500mg"]  
}}
```

입력: `\\"{text}\\"`

`"".strip()`

LLM 호출 및 **fallback** 처리

LLM 호출 실패 시 규칙 기반 요약으로 자동 **fallback**하여 서비스 안정성을 확보한다.

def summarize_text(text: str) -> dict:

text = normalize_text(text)

try:

```

        result = summarize_text_llm(text)

        if result.get("summary"):

            return result

    except Exception as e:

        print(f"[summary] llm failed, fallback: {e}")

    return {

        "summary": summarize_text_rule_based(text),

        "symptoms": [], "measurements": [], "medications": [],

    }

```

소수점 숫자(0.5 등)에서 문장이 잘리는 문제를 방지하기 위해 단순 `split(".")`이 아닌 정규식 분리를 사용한다.

```

def summarize_text_rule_based(text: str) -> List[str]:

    # 숫자 사이 소수점은 분리하지 않음: (?<!\d)\.(?!\\d)

    sentences = re.split(r'(?<!\d)\.(?!\\d)\s*[[!?]\s*', text)

    sentences = [s.strip() for s in sentences if s.strip()]

    return [s if s.endswith(".") else s + "." for s in sentences[:3]]

```

[Frontend]

AI 분석 결과 표시 및 저장 기능의 구현을 위하여 `ApiService` 모듈과 `api_models` 데이터 클래스를 활용하였고, `ResultScreen`을 구현하였다.

`ResultScreen`의 기능은 요약·용어 병렬 조회(F1), 요약 편집(F2), 진료과 선택 및 저장(F3)이고, F1은 `_fetchSummary()`와 `_fetchTerms()`를 각각 독립적인 `async` 메서드로 분리하여 `initState()`에서 동시에 호출하는 방법으로 구현되었으며, F2는 편집 버튼 탭 시 `_summary` 항목 수만큼 `TextEditingController`를 생성하여 각 항목을 `TextField`로 렌더링하고, 완료 탭 시 각 컨트롤러의 `.text.trim()`을 수집하여 빈 문자열을 제외한 후 `_summary`를 갱신하는 방법으로 구현되었으며, F3은 `showModalBottomSheet()`로 `_DepartmentPickerSheet`를 표시하고 반환값으로 `SaveRecordRequest`를 구성하는 방법으로 구현되었다. `ResultScreen`은 F1, F2, F3

기능을 수행하는데, F1의 처리 알고리즘은 `_loadingSummary`와 `_loadingTerms`를 독립적인 로딩 상태로 관리하여 먼저 완료된 섹션부터 즉시 표시하고, `warningMessage`가 있으면 STT 실패로 간주하여 F1 자체를 건너뛰는 것이다. F3의 처리 알고리즘은 바텀시트 `initState()`에서 `getDepartment()`를 호출하여 AI 추천 진료과를 가져오고, 추천값이 사전 정의된 14개 목록에 포함되면 기본 선택값으로 설정하며, 사용자가 최종 선택한 진료과와 날짜를 `Navigator.pop()`으로 반환받아 저장에 사용하는 것이다. 메모 입력이 있으면 '[메모] \$memo'를 요약 목록 끝에 삽입하여 함께 저장한다.

3.3.4 Q&A 기능

Q&A 기능의 구현을 위하여 `ApiService` 모듈을 활용하였고, `QuestionTab` 위젯을 구현하였다. `QuestionTab`은 `ResultScreen`과 `RecordDetailScreen` 양쪽에서 재사용된다.

`QuestionTab`의 기능은 질문 전송 및 AI 답변 표시(G1), 긴급 증상 표시(G2), 다음 진료 메모 관리(G3)이고, G1은 `TextEditingController`에 `addListener`를 등록하여 텍스트 변화를 감지하고 `_submittedQuestion`과 비교하여 전송 버튼 활성화 여부를 결정하는 방법으로 구현되었으며, G2는 `QuestionResponse.urgentSymptoms` 목록을 별도 `SectionCard`("지금 바로 확인 필요")에 렌더링하는 방법으로 구현되었으며, G3은 `_NextVisitItem {text, checked}` 내부 클래스의 리스트로 관리하는 방법으로 구현되었다. `QuestionTab`은 G1, G2, G3 기능을 수행하는데, G1의 처리 알고리즘은 전송 시 `_submittedQuestion = 현재텍스트로 저장하고 _buttonEnabled = false`로 설정하여 중복 제출을 방지하며, 이후 텍스트가 변경되면 `isDifferent = true`가 되어 버튼을 재활성화하고, `canAnswer == false`이면 `FloatingActionButton`("전송하기")을 `Stack`으로 우하단에 표시하는 것이다. G3의 처리 알고리즘은 + 버튼 탭 시 `showDialog()`로 텍스트 입력 다이얼로그를 표시하고 입력값으로 `_NextVisitItem`을 생성하여 리스트에 추가하며, `CheckboxListTile`로 렌더링하여 체크 시 `TextDecoration.lineThrough`와 회색 색상을 적용하는 것이다.

3.4 주요 기능의 구현

3.4.1 기능 1 - 음성 인식 및 교정 기능

기능 설명: 환자가 녹음한 진료 음성을 텍스트로 변환하고, 의료 용어 오인식을 자동 교정하여 정확한 전사 텍스트를 생성한다.

관련 모듈: `stt_google_streaming.py`, `stt_corrector.py`

[Backend]

데이터 흐름

Flutter 앱 (WAV 오디오)

| WebSocket 전송



`ws.py`: audio 메시지 수신

|



`GoogleStreamingSttBridge.recognize_once(raw_bytes)`

| WAV → PCM 변환



Google Cloud STT API 호출

| 전사 텍스트 반환



`postprocess_stt_text()`

- 약어 교정 (씨알피 → CRP)
- 단위 정규화 (밀리그램 → mg)
- 소수점 변환 (영점오 → 0.5)
- 간투사 제거

|



`correct_stt_text()`

- 규칙 기반 교정 (L4 LO → L4 LS)
- Gemini LLM 교정

|



WebSocket으로 교정된 텍스트 반환 → Flutter 앱

AI 인터페이스 구현 (Gemini STT 교정)

Gemini 2.5-flash를 STT 교정에 활용한다. temperature=0.1로 설정하여 교정 결과의 일관성을 극대화한다. 빈 응답이 반환될 경우 원문을 그대로 반환하도록 처리하여 교정 실패로 인한 데이터 손실을 방지한다.

```
def _build_correction_prompt(text: str) -> str:
```

```
    return f"""
```

너는 의료 **STT** 교정 시스템이다.

교정 규칙:

1. 척추 레벨 표기 교정 (LO -> L5, CO -> C6)
2. 알파벳/숫자 혼동 교정
3. 약어 오인식 교정 (씨알피 -> CRP)
4. 의미가 통하지 않는 단어만 교정
5. 원문 내용 추가/삭제 금지
6. 교정된 텍스트만 출력

```
원문: \"\"\"{text}\"\"\"
```

```
\"\"\".strip()
```

[Frontend]

STT 파이프라인 Flow

[사용자: 녹음 버튼 탭]



RecordingController.toggleListening()

└─ isListening == false → _startListening()

|

▼

AudioRecorder.startStream(pcm16bits, 16000Hz, 1ch)

stream.listen → _audioChunks.add(chunk)

transcriptStore.reset()

isListening = true → notifyListeners()

[사용자: 정지 버튼 탭]

|

▼

recorder.stop() + _audioSub.cancel()

Uint8List.fromList(_audioChunks.expand((x)=>x).toList())

base64Encode(bytes)

|

▼

WsTransport.sendJson({

type: 'audio', sessionId: 'test-session',

seq: DateTime.now().millisecondsSinceEpoch,

bytes: bytes.length, audioB64: <base64>

})

analyzing = true → notifyListeners()

|

▼

[백엔드 STT 처리]

|

└─ type == 'stt' 수신


```

| SttEvent.fromWs(e) → TranscriptStore.apply()
| _removeFiller() → isFinal 분기
| notifyListeners() → UI 갱신
| combinedText.isEmpty → analyzing = false
|
└─ type == 'warning' 수신

```

WarningEvent.fromWs(e) → transcriptStore.applyWarning()

analyzing = false → notifyListeners()

WebSocket 연결 초기화 Flow:

RecordingController._initWs()

|

▼

getWsUri() : 메모리 캐시 → SharedPrefs → 기본값

|

▼

WsTransport 생성 → connect()

WebSocketChannel.connect(uri)

→ connected → _sendSessionStart()

{type:'session.start', audio:{encoding, sampleRateHz, channels}}

→ 연결 끊김 → _scheduleReconnect()

delaySecs = min(2 << _retryCount, 30)

Timer → connect(reconnecting: true)

3.4.2 AI 기반 의료 요약 및 용어 설명 기능

기능 설명: 교정된 전사 텍스트를 Gemini로 분석하여 환자 친화적 요약을 생성하고, 등장하는 의료 용어에 대한 쉬운 설명을 함께 제공한다. 관련 모듈: `summarizer.py`, `term_extractor.py`, `routes/summary.py`, `routes/explain.py`

[Backend]

데이터 흐름

Flutter 앱 : POST /summary { "text" : "전사텍스트" }

|



`routes/summary.py`

`correct_stt_text(text, use_llm=True)` <- STT 교정 재적용

|



`summarize_text(corrected_text)`

|

Gemini API 호출



응답 파싱 : `summary / symptoms / measurements / medications`

|



SummaryResponse → Flutter 앱

Flutter 앱 : POST /explain { "text" : "전사텍스트" }

|



`routes/explain.py`

`correct_stt_text(text, use_llm=True)` <- STT 교정 재적용

|



```
extract_text(corrected_text)
```

```
| Gemini API 호출 (실패 시 medical_terms.json fallback)
```



TermItem 목록 반환 → Flutter 앱

AI 인터페이스 구현 (Gemini 요약)

요약 결과는 단순 문장 목록이 아니라 4개 필드로 구조화된 JSON으로 반환받는다.
이를 통해 Flutter 앱에서 요약/증상/수치/약물을 카테고리별로 표시할 수 있다.

```
def summarize_text_llm(text: str) -> dict:
```

```
    client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=_build_prompt(text),
        config=types.GenerateContentConfig(temperature=0.3),
    )
    return _parse_llm_summary(response.text)
```

응답 파싱 시 마크다운 코드블록(```json``) 제거 처리를 포함한다.

```
def _parse_llm_summary(raw_text: str) -> dict:
```

```
    raw_text = re.sub(r"```json\s*", "", raw_text)
    raw_text = re.sub(r"\s*```", "", raw_text)
    data = json.loads(raw_text)
    return {
        "summary": [s.strip() for s in data.get("summary", [])[:3],
        "symptoms": [s.strip() for s in data.get("symptoms", [])],
        "measurements": [s.strip() for s in data.get("measurements", [])],
```

```

        "medications": [s.strip() for s in data.get("medications", [])],
    }

```

Pydantic 응답 스키마

API 응답 구조는 Pydantic 모델로 정의하여 타입 안전성을 보장한다.

```

class StructuredInfo(BaseModel):

```

```

    symptoms: List[str] = []

```

```

    measurements: List[str] = []

```

```

    medications: List[str] = []

```

```

class SummaryResponse(BaseModel):

```

```

    summary: List[str]

```

```

    structured: Optional[StructuredInfo] = None

```

[Frontend]

AI 분석 결과 처리 Flow

[HomeScreen: 'AI 분석 결과 보기' 탭]

|



ResultScreen(transcript: combinedText)

initState()

└─ _fetchSummary() → POST /summary {text}

└─ _fetchTerms() → POST /explain {text} (동시 호출)

|



각 응답 수신 시 독립적으로 `setState()` → 해당 섹션 즉시 표시

[편집 버튼 탭]

`_initSummaryControllers(_summary)`

→ 각 항목을 `TextEditingController`로 생성

→ `TextField` 렌더링 (삭제 버튼, 항목 추가 버튼 포함)

[완료 탭]

→ `.text.trim().where(isNotEmpty)` → `_summary` 갱신

[저장하기 탭]



`showModalBottomSheet` → `_DepartmentPickerSheet`

`initState()` → `POST /department {text}`

AI 추천값 → `_selectedDept` 초기값 설정 + 'AI 추천' 배지

사용자: 진료과 칩 선택 / `showDatePicker()` 날짜 선택

저장하기 → `Navigator.pop((dept, date))`



`_doSave(dept, date)`

`memo.isNotEmpty` → `summaryToSave.add('[메모] $memo')`

`POST /records {date, department, clean_text, summary, terms}`

|



`SnackBar('기록이 저장되었습니다.')`

`Navigator.pushReplacement → RecordsScreen`

3.4.3 Q&A Flow

[사용자: 텍스트 입력]

`_onTextChanged()`

`hasContent = text.trim().isEmpty`

`isDifferent = (text != _submittedQuestion)`

`_buttonEnabled = hasContent && isDifferent`

[전송 버튼 탭]

|



`_submittedQuestion = 현재 텍스트`

`_buttonEnabled = false / _loading = true`

|



`POST /question {question, context: transcript}`

|

`canAnswer == true`

| Q. 질문 + 좌측 파란 테두리 답변 박스 표시

|

|— canAnswer == false

| "AI가 답변할 수 없어요" + FloatingActionButton 표시

|

|— urgentSymptoms.isNotEmpty

'지금 바로 확인 필요' 섹션에 빨간 경고 아이콘과 함께 목록 표시

[다음 진료 메모: + 버튼 탭]

showDialog → TextField 입력

→ _nextVisitItems.add(_NextVisitItem(text))

→ CheckboxListTile: 체크 시 취소선 + 회색 처리

미진 사항

term_extractor 모듈의 Gemini 호출 실패 시 fallback으로 사용하는

medical_terms.json은 현재 내과, 정형외과, 심장내과, 신경과 기준 60개 항목을 수록하고 있다. 향후 진료과 자동 감지 기능 구현 시 해당 JSON을 진료과별로 분리하여 관련도 높은 용어를 우선 제공하는 방향으로 확장할 예정이다.

또한 진료과 자동 추출 서비스(department_extractor.py)는 현재 미구현 상태이며, STT 텍스트에서 Gemini로 진료과를 추론하는 기능을 추가 개발 예정이다.

3.5 기타 — AI 활용 상세

본 앱은 서버 측 AI를 총 4개의 엔드포인트로 활용하며, 클라이언트는 STT로 변환된 텍스트를 입력으로 전달하고 AI의 분석 결과를 수신하여 표시한다. AI 모델 자체는 서버에서 운용되며, 클라이언트는 HTTP REST API를 통해 결과만을 수신한다.

진료 내용 요약 기능의 구현을 위하여 /summary 엔드포인트를 활용하였다. 요청 형식은 {"text": <STT 전체 텍스트>}이고, 응답 형식은 {"summary": ["항목1", "항목2", ...]}이다. 클라이언트는 SummaryResponse.fromJson()으로 List<String>을 역직렬화하여 **BulletList** 위젯에 바인딩하며, 사용자가 각 항목을 직접 편집하거나 삭제·추가한 결과를 저장 시 그대로 서버에 전송한다. 추가 메모가 있으면 '[메모] \$memo' 형태로 요약 목록 끝에 삽입하여 함께 저장한다.

의료 용어 추출 기능의 구현을 위하여 /explain 엔드포인트를 활용하였다. 요청 형식은 {"text": <STT 전체 텍스트>}이고, 응답 형식은 {"terms": [{"term": "용어명", "description": "설명"}, ...]}이다. 클라이언트는 ExplainResponse.fromJson()으로 List<MedTerm>을 역직렬화하며, TermChipList 위젯에서 각 항목을 ActionChip으로 렌더링하고, 탭 시 AlertDialog로 설명을 표시한다. 저장 시에는 MedTerm.toJson()으로 직렬화하여 서버에 그대로 전송한다.

진료과 자동 분류 기능의 구현을 위하여 /department 엔드포인트를 활용하였다. 요청 형식은 {"text": <STT 전체 텍스트>}이고, 응답 형식은 {"department": "진료과명"}이다. 클라이언트는 반환된 진료과명이 사전 정의된 14개 목록(내과, 정형외과, 피부과, ...)에 포함되면 기본 선택값으로 설정하고 'AI 추천' 배지를 표시하며, 목록에 없는 값이면 기본값(내과)을 유지한다. AI 추천값은 참고용이며 최종 결정은 사용자가 칩 선택으로 수행한다.

AI Q&A 기능의 구현을 위하여 /question 엔드포인트를 활용하였다. 요청 형식은 {"question": <사용자 질문>, "context": <STT 전체 텍스트>}이고, 응답 형식은 {"can_answer": true/false, "answer": "...", "urgent_symptoms": ["증상1", ...]}이다. context 필드로 진료 원문을 함께 전달하여 AI가 진료 내용 범위 내에서 답변하도록 한다. 클라이언트는 canAnswer 값에 따라 답변 표시 또는 "의사에게 전송하기" 버튼 표시로 분기하며, urgentSymptoms는 canAnswer 결과와 무관하게 항상 별도 섹션에 빨간 경고 아이콘과 함께 표시한다.